Exercises · AI Frameworks

AI07 Exercises

Work each one before reading its solution. Practise Exercise 6 out loud.

Exercise 1: Choose a stack

For each team, recommend an option, and name one thing to check.

(a) Java and Spring Boot 4, needing RAG over Postgres and two model providers. **(b)** Python, needing a three-step approval flow over days. **(c)** Plain Java, one internal agent, wants the newest Claude features at once. **(d)** A small team with no infrastructure staff, wanting scheduled overnight agents. **(e)** TypeScript, building an internal code-migration agent.

Solution

(a) Spring AI 2.0.x. Check: does it support the current model and caching as you need?

(b) LangGraph with a Postgres checkpointer. Check: side effects around `interrupt()`.

(c) `anthropic-java`, using the Tool Runner or a manual loop. Check: runner features are in beta.

(d) Managed Agents, using scheduled deployments. Check: it's in beta, and consider data and platform limits.

(e) Agent SDK. Check: permissions and hooks for what it may touch.

Exercise 2: Fix the node

```
def approve(state):
    db.insert_audit("approval requested", state["id"])
    send_slack("Please approve " + state["id"])
    try:
        answer = interrupt({"id": state["id"]})
    except Exception:
        answer = False
    if answer:
        payments.charge(state["id"])
    return {"approved": answer}
```

(a) What goes wrong? **(b)** Rewrite it.

Solution

(a)

1. On resume, the node restarts, so the audit row and Slack message are repeated.
2. The bare `except Exception` catches the pause, per the docs, so `answer` may become `False`, and the graph won't pause correctly.
3. `charge` isn't protected against repeats, so a crash after charging and before the checkpoint could repeat it.

(b)

```
def notify(state):
    db.upsert_audit("approval requested", state["id"])  # an earlier node, won't re-run
    send_slack("Please approve " + state["id"])
    return {}

def approve(state):
    return {"approved": interrupt({"id": state["id"]})}

def charge(state):
    if state["approved"]:
        payments.charge(state["id"], idempotency_key=state["id"])
    return {}
```

A separate `notify` node isn't repeated by resuming `approve`, though a crash retry could still repeat it, which is why the audit is an upsert, and why Slack should tolerate a duplicate.

Exercise 3: Audit a default

Your team's Spring AI service has this:

```
@Tool(description = "Find a customer")
Customer find(String tenantId, String email) {
    return jdbc.queryForObject(SQL, mapper, tenantId, email);
}
```

(a) List the problems. **(b)** Rewrite the signature. **(c)** What's sent to the model when the query fails? **(d)** What test would catch this?

Solution

(a) The description is too short (AI06 L3). `tenantId` is filled in by the model. `email` has no format or description. No result from `queryForObject` throws, so "no match" looks like a failure. Database errors go to the model with raw messages.

(b)

```
@Tool(description = """
        Finds one customer in the caller's tenant by exact email. Use this first \
        when a customer gives an email. Returns no match, rather than an error, \
        if none exists.""")
CustomerResult find(@ToolParam(description = "Exact email") String email,
        ToolContext context) {
```

The tenant is read from `context`, set by your code from the authenticated request.

(c) By default, the exception's message, which may include SQL or connection details.

(d) A scripted-model test: make the data source throw an exception containing a marker string, and assert that the marker never appears in the next request to the model. Add one that asserts `tenantId` isn't in the tool's schema.

Exercise 4: Write the evaluation

You're evaluating a new framework, X. Using this module's checklist, write five tests or checks, with what would make you reject it.

Solution

1. **Status.** Read the repository: release dates and security issues. Reject: archived, or no release for a long time.
2. **Recorded requests.** A recording model, and read the prompt. Reject: you can't see or change prompts.
3. **Runaway.** A model that always calls a tool. Reject: no limit, or a limit reported as success that you can't detect.
4. **Error leak.** A tool throwing a marker. Reject: no way to stop it reaching the model.
5. **Your model.** One real call on your exact model with your features: caching, tools and effort. Reject: a key parameter can't be set, and there's no way through to the raw request.

Also count framework imports needed in business code, to judge how hard leaving would be.

Exercise 5: Read the lab output

```
request 1: SYSTEM USER
request 2: SYSTEM USER AI TOOL_EXECUTION_RESULT
request 3: SYSTEM USER AI TOOL_EXECUTION_RESULT AI USER
```

Memory uses `MessageWindowChatMemory.withMaxMessages(10)`.

(a) How many model calls did the two user messages cost? **(b)** Assume each message averages 300 tokens, and calculate input tokens per request, ignoring tools and caching. **(c)** A long conversation reaches 10 stored messages, then more arrive. What's the risk? **(d)** How would you keep an order amount from being lost?

Solution

(a) 3 calls: 2 for the first message, because of the tool, and 1 for the second.

(b) Request 1 has 2 messages: 600 tokens. Request 2 has 4: 1,200. Request 3 has 6: 1,800. **Total: 3,600 input tokens.**

(c) Older messages get evicted, so the agent can forget early facts, such as the amount. A window may also separate a tool call from its result, so check how your framework handles that.

(d) A case facts block outside the window (TS 5.1), held by your code and added to each request, for example through the system message provider. Keep real records in your database.

Exercise 6: Defend a decision

A staff engineer says: "Frameworks are a mistake. We should ban them, and only use the raw SDK. Look at everything your labs found."

The company has 12 teams building AI features, mostly with Spring Boot. Respond in about a minute.

Model answer

"The labs do show real risks, but they were all things we could find and fix in an afternoon: raw errors going to the model, and a limit that's returned as text. So the lesson is to test and configure, not to ban.

With 12 teams on the raw SDK, each will write its own memory, tool schemas, retries and vector store code, and they'll make those same mistakes, separately. Anthropic's advice is to start with the API and understand what a framework does, which isn't the same as never using one.

My proposal: an internal Spring starter, wrapping one framework with safe defaults for errors, limits and logging, behind a small port. Teams with a need that the framework can't meet can use the SDK inside the adapter. A nightly test runs on our real models.

I'd reconsider if we found ourselves working around the framework constantly, or if a team's main need was the newest Claude features. At that point, the SDK with a shared library might be better. And I'd want to measure that, rather than guess."